Git Collaboration Guidelines v1 - Bogosort Team

To ensure smooth collaboration and maintain code quality, we will follow a structured Git workflow using branches, pull requests, and reviews. The `main` branch should always remain stable, so direct commits to `main` are not allowed. All changes must be made through feature branches and merged via Pull Requests.

Key Collaboration Rules

- **Do not commit directly to `main`.**
- **Always create a new branch for each task.**
- **Use Pull Requests for merging.**
- **At least one team member must review and approve each PR before merging.**
- **Keep commit messages and PR descriptions clear and descriptive.**

---

1. Branches

All development should happen in separate branches, not directly in `main`.

Use descriptive names with prefixes:

- `feature/` for new features
  Example: `feature/user-login`
- `bugfix/` for bug fixes
  Example: `bugfix/token-expiry`
- `docs/` for documentation updates
  Example: `docs/readme-update`

Step 1: Clone the Repository

Each team member should clone the repository locally.

```
git clone <repository-url>
cd <repository-name>
```

Step 2: Ensure Your Main Branch Is Up to Date

Before starting any work, update your local `main` branch.

```
git checkout main
git pull origin main
```

Step 3: Create a New Branch

Create a new branch for the task you are working on.

Example:

```
git checkout -b feature/dashboard-visualisation
```

This creates a new branch and switches you to it.

---

2. Making Changes and Committing

Once you are in your branch, you can edit files and commit your changes.

Step 1: Check file changes

```
git status
```

This shows which files were modified.

Step 2: Add files to the commit

To add a specific file:

```
git add filename.py
```

Or add all changed files:

```
git add .
```

Step 3: Commit the changes

Create a commit with a clear message describing the change.

```
git commit -m "Add dashboard visualisation component"
```

Step 4: Push the Branch to GitHub

After committing locally, push your branch to GitHub.

```
git push origin feature/dashboard-visualisation
```

---

3. Creating a Pull Request (PR)

A Pull Request is a request to merge your branch into `main`. This allows other team members to review your work before it becomes part of the main project.

Step 1: Open GitHub Repository

Go to the repository page on GitHub. GitHub will usually display a banner suggesting:

> "Compare & pull request"

Click it.

Alternatively:

- Click **Pull Requests**
- Click **New Pull Request**

Step 2: Select branches

Set:

- **Base branch:** `main`
- **Compare branch:** your feature branch

Step 3: Write a clear PR description

Include:

- What was implemented
- What the change does
- Which issue it solves
- Any important notes

Example:

> This PR adds the interactive dashboard visualisation.
> Changes include:
>
> - New visualisation component
> - API call for model predictions
> - Minor UI improvements
>
> No breaking changes expected.

---

4. PR Review Process

All Pull Requests should be reviewed before merging.

The PR reviewer should:

- Checking code quality and logic
- Ensuring coding standards are followed
- Identifying potential bugs or conflicts
- Requesting changes if needed
- Approving the PR once it is ready

Possible review outcomes

The reviewer can:

- **Approve**
  → The PR is ready to merge.
- **Request Changes**
  → The author must modify the code before merging.
- **Comment**
  → Suggestions or clarifications.

---

5. Making Changes After Review

If the reviewer asks for changes:

1. Edit the code locally
2. Commit the fixes

```
git add .
git commit -m "Fix issues from PR review #number"
```

3. Push again

```
git push origin feature/dashboard-visualisation
```

The PR updates automatically.

---

6. Merging the Pull Request

Once the PR is approved, it can be merged into `main`.

Step 1: Merge on GitHub

Click **Merge Pull Request**.

---

7. Updating Your Local Repository After Merge

After a PR is merged, update your local `main` branch.

```
git checkout main
git pull origin main
```

This ensures your local code is synchronised with the latest changes.

---

8. Handling Conflicts

Conflicts may occur if multiple branches modify the same code. This usually occurs when:

- multiple people modify the same file
- multiple people modify the same lines of code
- One branch deletes a file while another modifies it

GitHub will show a merge conflict warning in the PR.

8.a Resolving conflicts locally

1. Pull the latest `main` branch
2. Merge it into your branch
3. Check conflicted files

```
git checkout feature/dashboard-visualisation
git fetch origin
git merge origin/main
```

4. Resolve conflicts manually
5. Commit the fixes
6. Push again

Example:

```
git add backend/api.py
git commit -m "Resolve merge conflict with main"
git push origin feature/dashboard-visualisation
```

8.b Resolving conflicts directly on GitHub

Sometimes GitHub allows resolving conflicts in the PR interface.

Steps:

1. Open the Pull Request
2. GitHub shows “This branch has conflicts”
3. Click **Resolve conflicts**
4. Edit the code in the browser
5. Mark as resolved
6. Commit the merge

However, this only works for simple conflicts.

---

9. Forks

Forking creates a personal copy of the repository under your GitHub account.

Since we already have write access to the project repository, forking is not required for our workflow. Instead, we will work directly in the shared repository using feature branches.